



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E
TECNOLOGIA DO RIO GRANDE DO NORTE – IFRN

CARLA GABRIELE DA SILVA SANTOS
DAVID GABRIEL DUTRA SOUZA
EMMILY KETTILY BEZERRA DA SILVA
GUILHERME DAVID ARAÚJO OLIVEIRA
INGRID MONTEIRO DE MELO

**RELATÓRIO DE TESTES UNITÁRIOS, DE
INTEGRAÇÃO E DE SISTEMA NO
DESENVOLVIMENTO DO IFSPORTS**

CAICÓ/RN
2026

CARLA GABRIELE DA SILVA SANTOS
DAVID GABRIEL DUTRA SOUZA
EMMILY KETTILY BEZERRA DA SILVA
GUILHERME DAVID ARAÚJO DE OLIVEIRA
INGRID MONTEIRO DE MELO

IFSPORTS: SISTEMA WEB PARA
INCENTIVO À PRÁTICA DE ESPORTES NO
IFRN

CAMPUS CAICÓ

Orientador: Romerito Campos de Andrade
Coorientador: Hudson Pablo de Oliveira Bezerra

CAICÓ, RN
2026

1 INTRODUÇÃO

O IFSports é uma plataforma web de gestão esportiva desenvolvida para o IFRN Campus Caicó, com o objetivo de substituir processos manuais e aplicativos de mensagens pela centralização digital da organização de treinos escolares. O sistema permite que professores criem e gerenciem treinos, definam horários e limites de vagas, enquanto estudantes podem visualizar treinos disponíveis, inscrever-se, confirmar presença por meio de check-in e cancelar inscrições. Além disso, o sistema conta com autenticação, validação de e-mail institucional, controle de papéis de usuário e um módulo de gamificação com pontuação, níveis e conquistas.

Diante da complexidade de regras de negócio presentes em um sistema com múltiplos fluxos (autenticação, inscrição, check-in, validação de frequência, pontuação e conquistas), tornou-se necessária a implementação de uma suíte de testes automatizados capaz de garantir que essas regras funcionem conforme o esperado. Este relatório documenta o processo de elaboração, execução e análise dos testes unitários, de integração e de sistema desenvolvidos para o IFSports.

2 OBJETIVOS

2.1 Objetivo Geral

Garantir a qualidade e a confiabilidade das principais regras de negócio do sistema IFSports por meio da criação de testes automatizados, unitários e de integração, e da validação manual de fluxos completos por meio de teste de sistema, cobrindo as camadas de formulários, modelos, serviços, utilitários, rotas e a experiência do usuário final.

2.2 Objetivos Específicos

- Verificar isoladamente o comportamento de funções, propriedades de modelos e serviços da aplicação (testes unitários);
- Validar o funcionamento de fluxos completos por meio de requisições simuladas às rotas Flask, incluindo efeitos no banco de dados (testes de integração);
- Demonstrar, por meio de gravação de tela, o funcionamento ponta a ponta dos fluxos críticos sob a perspectiva do usuário real (teste de sistema);
- Identificar inconsistências, falhas de lógica e comportamentos não documentados a partir da elaboração dos testes;

- Documentar a cobertura obtida, os principais achados do processo e as lacunas que ainda demandam atenção futura.

3 FUNDAMENTAÇÃO TEÓRICA

Testes de software são procedimentos sistemáticos utilizados para verificar se um sistema se comporta conforme o especificado, permitindo identificar defeitos antes que cheguem ao ambiente de produção. Dentro da chamada "pirâmide de testes", destacam-se três níveis centrais neste trabalho: os testes unitários, os testes de integração e os testes de sistema.

Os testes unitários verificam a menor unidade testável de um sistema de forma isolada — uma função, um método ou uma propriedade de uma classe — sem depender de componentes externos como banco de dados real, rede ou outros módulos. Os testes de integração verificam a interação entre diferentes partes do sistema em conjunto, como, por exemplo, uma requisição HTTP completa que passa pela rota, pelo formulário, pelo banco de dados e retorna uma resposta ao usuário. Já os testes de sistema avaliam a aplicação em execução como um todo, sob a perspectiva do usuário final, verificando se as jornadas completas — que muitas vezes atravessam vários dos componentes já testados isoladamente — realmente funcionam quando integradas em um cenário de uso real.

Para a automatização dos testes unitários e de integração, foi utilizado o pytest, framework de testes para Python amplamente adotado por sua sintaxe simples baseada em funções, seu sistema de fixtures (mecanismo de preparação e injeção de dependências de teste) e sua extensibilidade por meio de plugins, como o pytest-flask (integração com a aplicação Flask) e o pytest-mock (substituição de dependências externas por objetos simulados, os chamados mocks). Já o teste de sistema foi conduzido por meio de gravação de tela do sistema em execução, sem o uso de ferramentas de automação de navegador (como Selenium ou Playwright)

4 METODOLOGIA

4.1 Ferramentas e Ambiente de Testes

A suíte de testes automatizados do IFSports foi construída sobre a seguinte base tecnológica:

- pytest — executor e framework de testes;
- pytest-flask — fornece a integração entre o pytest e a aplicação Flask, possibilitando o uso do cliente de testes (test client) e do contexto de aplicação;

- `pytest-mock` — permite substituir, durante os testes, funções que produzem efeitos colaterais externos (como emissão de eventos via `Socket.IO` e criação de notificações), evitando que o teste dependa desses serviços;
- `Flask-SQLAlchemy` — o banco de dados de testes é criado e destruído automaticamente a cada execução da suíte, garantindo que os testes não interfiram nos dados de produção nem entre si.

Para reduzir a duplicação de código entre os testes, foram adotados dois padrões auxiliares: as `fixtures` compartilhadas (`tests/fixtures.py`), responsáveis por fornecer objetos básicos já persistidos no banco de testes, e as `factories` (`tests/factories.py`), funções que centralizam a criação de instâncias específicas com valores customizáveis.

4.2 Organização da Suíte de Testes

Os testes automatizados foram organizados em subpastas conforme a camada da aplicação exercitada:

- `tests/forms/` — validação de formulários (`WTForms`);
- `tests/models/` — propriedades e métodos dos modelos de dados (`SQLAlchemy`);
- `tests/utils/` — funções utilitárias puras ou de apoio à lógica de negócio;
- `tests/services/` — regras de negócio centralizadas em serviços, como gamificação e envio/validação de tokens de e-mail;
- `tests/integration/` — testes de integração que exercitam rotas completas por meio do cliente de testes do `Flask`.

4.3 Estratégia de Isolamento e Determinismo

- Congelamento de tempo: funções sensíveis à data e hora atual foram testadas substituindo-se temporariamente a referência de tempo do sistema por um valor fixo, evitando testes intermitentes;
- Mocks de efeitos colaterais: dependências externas (notificações, eventos em tempo real) foram substituídas por mocks nos testes de rota;
- Banco de dados isolado por execução: a suíte utiliza um banco de dados dedicado a testes, recriado a cada execução e limpo entre os casos de teste.

5 TESTES IMPLEMENTADOS

A Tabela 1 resume a suíte de testes automatizados, organizada por arquivo e camada da aplicação. Os casos individuais de cada arquivo são detalhados na íntegra no Apêndice (seção 9).

Tabela 1 — Resumo da suíte de testes automatizados

Camada	Arquivo	Nº de testes	Foco principal
Formulário	test_auth_forms.py	4	Validação de login e dados de acesso
Formulário	test_register_form.py	5	Validação de cadastro e domínio de e-mail institucional
Formulário	test_treino_form.py	6	Validação de criação de treino (fixo x avulso)
Modelo	test_treino_model.py	14	Propriedades do treino (vagas, ocorrência aberta, check-in)
Modelo	test_treino_ocorrencia_model.py	5	Vagas restantes e status de validação da ocorrência
Modelo	test_frequencia_model.py	4	Verificação de frequência já validada
Utilitário	test_treino_utils.py	8	Cálculo de datas de treino e obtenção de ocorrência aberta
Serviço	test_email_service.py	3	Geração e validação de tokens de e-mail
Serviço	test_gamification.py	11	Cálculo de nível, progresso e pontuação do usuário
Serviço	test_verificar_conquistas.py	17	Regras de desbloqueio de conquistas
Integração	test_auth_routes.py	10	Login, confirmação de e-mail, redefinição de senha e permissões
Integração	test_checkin.py	6	Fluxo completo de check-in em um treino
Integração	test_treino_crud.py	5	Criação, edição e remoção de treinos
Integração	test_validar_frequencias.py	7	Validação de frequência pelo professor

Ao todo, a suíte automatizada reúne 105 casos de teste: 77 unitários e 28 de integração, distribuídos em 14 arquivos.

5.1 Testes Unitários

Os testes unitários cobrem a validação de formulários (WTForms), as propriedades calculadas dos modelos de domínio (Treino, TreinoOcorrencia, Frequencia), as funções utilitárias de cálculo de datas de treinos recorrentes, a geração e validação de tokens de e-mail, e a lógica de gamificação — cálculo de nível, progresso e, com maior profundidade, as regras de desbloqueio automático de conquistas, que consideram múltiplos critérios combinados (inscrições, presenças, avaliações, média de notas e posição no ranking), incluindo o comportamento recursivo pelo qual o bônus de uma conquista pode desbloquear outra na mesma verificação.

5.2 Testes de Integração

Os testes de integração exercitam o fluxo completo de uma requisição HTTP por meio do cliente de testes do Flask, cobrindo autenticação (login, confirmação de e-mail, redefinição de senha), autorização (bloqueio de rotas exclusivas de professor), o ciclo de vida de um treino (criação, edição e remoção, incluindo tentativas de acesso indevido entre professores) e o fluxo de check-in e validação de frequência, verificando tanto o efeito no banco de dados quanto as regras de autorização envolvidas.

6 TESTE DE SISTEMA

O teste de sistema foi conduzido por meio da gravação de tela do sistema em execução, navegando manualmente pelos fluxos críticos já cobertos pelos testes unitários e de integração, porém demonstrados de ponta a ponta sob a perspectiva do usuário real. Optou-se por não configurar ferramentas de automação de navegador (Selenium ou Playwright) em razão do prazo disponível para a entrega deste trabalho; a validação manual gravada em vídeo foi considerada suficiente para comprovar o funcionamento dos cenários de maior risco, sem a pretensão de repetir a cobertura já garantida pelos 105 testes automatizados.

O vídeo, com duração de aproximadamente 4 minutos e 30 segundos, percorre os seguintes cenários:

- Cadastro com e-mail fora do domínio institucional (rejeitado) e com e-mail institucional válido (aprovado), seguido de confirmação de conta;
- Login com senha incorreta (rejeitado) e com credenciais corretas (autenticado);
- Criação de um treino avulso por um professor;
- Criação de um treino fixo semanal, inscrição e check-in de um aluno, validação da frequência pelo professor e verificação de que uma nova ocorrência é gerada automaticamente para a semana seguinte;
- Inscrição e check-in de um aluno em um treino, com verificação do acréscimo de pontos;
- Validação de frequência pelo professor, com verificação de pontos e desbloqueio de conquista;
- Tentativa de acesso de um aluno a uma rota exclusiva de professor (bloqueio)
- Edição de um treino pelo próprio professor e tentativa de edição de um treino pertencente a outro professor (bloqueada).

Cada cenário do vídeo foi identificado por um card de texto no formato "Ação / Esperado", de modo a registrar explicitamente o comportamento esperado antes da demonstração do

resultado — prática adotada para preservar o caráter de teste da gravação, e não apenas de demonstração do sistema.

Vídeo do teste de sistema:



Link: <https://youtu.be/1OaI8bJtMMU>

7 RESULTADOS E DISCUSSÃO

7.1 Cobertura Obtida

A suíte desenvolvida cobre, com boa profundidade, os fluxos considerados de maior risco para a integridade dos dados, a segurança de acesso e a experiência do usuário: autenticação e recuperação de conta, autorização por papel de usuário, o ciclo de vida de treinos, o fluxo de check-in e validação de frequência, e o sistema de gamificação — este último especialmente relevante por concentrar a lógica de negócio mais complexa do sistema.

Todos os 105 testes automatizados foram executados com sucesso na versão final do projeto, conforme evidenciado na Figura 1.

Figura 1 — Saída do terminal com a execução completa da suíte de testes

```
(envtests) C:\Users\david\Documents\IFSports-PP>python -m pytest tests/ --disable-warnings -q
===== test session starts =====
platform win32 -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\david\Documents\IFSports-PP
configfile: pytest.ini
plugins: anyio-4.13.0, flask-1.3.0, mock-3.15.1
collected 105 items

tests\forms\test_auth_forms.py ..... [ 3%]
tests\forms\test_register_form.py ..... [ 8%]
tests\forms\test_treino_form.py ..... [ 14%]
tests\integration\test_auth_routes.py ..... [ 23%]
tests\integration\test_checkin.py ..... [ 29%]
tests\integration\test_treino_crud.py ..... [ 34%]
tests\integration\test_validar_frequencias.py ..... [ 40%]
tests\models\test_frequencia_model.py ..... [ 44%]
tests\models\test_treino_model.py ..... [ 58%]
tests\models\test_treino_ocorrencia_model.py ..... [ 62%]
tests\services\test_email_service.py ..... [ 65%]
tests\services\test_verification.py ..... [ 76%]
tests\services\test_verificar_conquistas.py ..... [ 92%]
tests\utils\test_treino_utils.py ..... [100%]

===== 105 passed, 86 warnings in 21.61s =====
```

7.2 Achados Relevantes Durante o Desenvolvimento dos Testes

A elaboração dos testes não apenas validou o comportamento esperado do sistema, mas também revelou pontos que mereceram atenção especial — dois deles, inclusive, corrigidos no próprio código-fonte ao longo do desenvolvimento deste trabalho:

- Campo de papel provisório no cadastro: o campo de seleção "Você é: Aluno/Professor", exibido no formulário de cadastro, não é utilizado pela rota de registro para determinar o tipo de conta criada — essa definição é feita exclusivamente a partir do domínio do e-mail informado. Achado registrado para avaliação futura quanto à intenção original do campo;
- Assimetria de autorização entre edição e remoção de treino: identificou-se, em uma versão anterior do sistema, que a rota de remoção de treino (`remover()`) verificava se o treino pertencia ao professor autenticado, retornando erro 404 em caso negativo, mas a rota de edição (`editar()`) não realizava essa mesma verificação — permitindo que qualquer professor autenticado editasse o treino de outro professor. O achado foi registrado no caso de teste TI19 e, entre a identificação do problema e a entrega deste trabalho, a inconsistência foi corrigida no código-fonte: a rota `editar()` passou a aplicar a mesma verificação de propriedade já existente em `remover()`. O caso de teste TI19 foi atualizado para validar o comportamento corrigido (bloqueio com erro 404), demonstrando o ciclo completo de valor dos testes automatizados: detecção de uma falha real, correção no código e confirmação automatizada da correção;

- Escopo de sessão do SQLAlchemy em testes: durante a construção dos testes de modelo, identificou-se um erro decorrente do uso de um contexto de aplicação aninhado dentro de testes que já eram executados sob um contexto único mantido ativo durante toda a suíte. A correção consistiu em remover o contexto redundante, alinhando os testes ao padrão já utilizado no restante do projeto.

7.3 Lacunas de Cobertura Ainda Existentes

Em razão do prazo disponível, permanecem sem testes automatizados os controllers de avisos, FAQ, modalidades, mural, notificações, ranking e perfil de usuário, todos de baixo risco relativo por concentrarem operações de CRUD simples e leitura, sem regras de negócio comparáveis às já cobertas. Essas lacunas são apontadas como sugestão de continuidade para trabalhos futuros.

8 DIFICULDADES ENCONTRADAS

- Equilíbrio entre abrangência e prazo: foi necessário priorizar criteriosamente quais módulos testar primeiro, optando pelos fluxos de maior risco em vez de buscar cobertura total do sistema;
- Particularidades de escopo de sessão do ORM: a compreensão do funcionamento do escopo de sessão do SQLAlchemy dentro do ciclo de vida dos testes exigiu atenção especial para evitar tanto falsos negativos quanto falsos positivos;
- Validação da lógica de negócio antes da automação: por se tratar de regras com múltiplas condições combinadas, foi necessário um processo cuidadoso de leitura do código-fonte e simulação manual dos cenários antes da escrita de cada teste;

9 CASOS DE TESTE (APÊNDICE)

Esta seção apresenta a relação completa dos casos de teste unitários e de integração da suíte automatizada, com identificador, objetivo e resultado obtido na execução final.

9.1 Casos de Teste Unitários

Tabela 2 — Casos de teste unitários (TU01–TU77)

ID	Tipo	Objetivo	Resultado
TU01	Unitário	Validar cadastro com e-mail de domínio de aluno	Aprovado
TU02	Unitário	Validar cadastro com e-mail de domínio de professor	Aprovado
TU03	Unitário	Rejeitar e-mail fora do domínio institucional	Aprovado
TU04	Unitário	Rejeitar cadastro com senhas diferentes	Aprovado
TU05	Unitário	Validar papel de aluno pelo domínio do e-mail	Aprovado
TU06	Unitário	Validar papel de professor pelo domínio do e-mail	Aprovado

TU07	Unitário	Rejeitar e-mail não institucional	Aprovado
TU08	Unitário	Rejeitar senha e confirmação divergentes	Aprovado
TU09	Unitário	Rejeitar senha abaixo do tamanho mínimo	Aprovado
TU10	Unitário	Validar treino avulso com data informada	Aprovado
TU11	Unitário	Validar treino fixo com dia e horário	Aprovado
TU12	Unitário	Rejeitar treino fixo sem horário informado	Aprovado
TU13	Unitário	Rejeitar treino avulso sem data informada	Aprovado
TU14	Unitário	Rejeitar quantidade de vagas menor que 1	Aprovado
TU15	Unitário	Rejeitar quantidade de vagas inválida	Aprovado
TU16	Unitário	Retornar falso sem frequências registradas	Aprovado
TU17	Unitário	Retornar falso com frequência pendente	Aprovado
TU18	Unitário	Retornar verdadeiro com frequência validada	Aprovado
TU19	Unitário	Isolar a validação por ocorrência	Aprovado
TU20	Unitário	Retornar nulo sem ocorrências cadastradas	Aprovado
TU21	Unitário	Retornar a ocorrência ativa aberta	Aprovado
TU22	Unitário	Ignorar ocorrência já validada	Aprovado
TU23	Unitário	Ignorar ocorrência inativa	Aprovado
TU24	Unitário	Retornar zero vagas ocupadas sem ocorrência	Aprovado
TU25	Unitário	Calcular vagas ocupadas da ocorrência aberta	Aprovado
TU26	Unitário	Usar a quantidade total sem ocorrência aberta	Aprovado
TU27	Unitário	Descontar vagas ocupadas do total	Aprovado
TU28	Unitário	Retornar nulo sem data de exibição	Aprovado
TU29	Unitário	Exibir a data da ocorrência aberta	Aprovado
TU30	Unitário	Retornar falso para usuário professor	Aprovado
TU31	Unitário	Retornar falso sem ocorrência aberta	Aprovado
TU32	Unitário	Retornar falso sem inscrição do aluno	Aprovado
TU33	Unitário	Retornar verdadeiro com inscrição do aluno	Aprovado
TU34	Unitário	Retornar vagas totais sem ocupação	Aprovado
TU35	Unitário	Descontar ocupação das vagas restantes	Aprovado
TU36	Unitário	Retornar zero vagas quando lotado	Aprovado
TU37	Unitário	Retornar falso antes da validação	Aprovado
TU38	Unitário	Retornar verdadeiro após validação	Aprovado
TU39	Unitário	Gerar e validar token de e-mail	Aprovado
TU40	Unitário	Rejeitar token expirado	Aprovado
TU41	Unitário	Rejeitar token adulterado	Aprovado
TU42	Unitário	Calcular nível amador pela pontuação	Aprovado
TU43	Unitário	Calcular nível dedicado pela pontuação	Aprovado
TU44	Unitário	Calcular nível competidor pela pontuação	Aprovado
TU45	Unitário	Calcular nível de atleta de alto nível	Aprovado
TU46	Unitário	Calcular nível de atleta profissional	Aprovado

TU47	Unitário	Calcular progresso no início do nível	Aprovado
TU48	Unitário	Calcular progresso na metade do nível	Aprovado
TU49	Unitário	Calcular progresso no segundo nível	Aprovado
TU50	Unitário	Calcular progresso no nível máximo	Aprovado
TU51	Unitário	Adicionar pontos ao usuário	Aprovado
TU52	Unitário	Remover pontos do usuário	Aprovado
TU53	Unitário	Conceder conquista na primeira vez	Aprovado
TU54	Unitário	Impedir duplicação de conquista já concedida	Aprovado
TU55	Unitário	Conceder conquista ao atingir meta de pontos	Aprovado
TU56	Unitário	Não conceder conquista abaixo da meta	Aprovado
TU57	Unitário	Contar inscrições independente do status	Aprovado
TU58	Unitário	Ignorar inscrições sem presença confirmada	Aprovado
TU59	Unitário	Contar apenas frequências com presença	Aprovado
TU60	Unitário	Conceder conquista na primeira avaliação recebida	Aprovado
TU61	Unitário	Exigir nota máxima para conquista de nota perfeita	Aprovado
TU62	Unitário	Conceder conquista com nota máxima	Aprovado
TU63	Unitário	Exigir mínimo de três avaliações para a média	Aprovado
TU64	Unitário	Conceder conquista de média com meta atingida	Aprovado
TU65	Unitário	Exigir dez avaliações para conquista de elite	Aprovado
TU66	Unitário	Exigir mínimo de dez participantes no ranking	Aprovado
TU67	Unitário	Conceder conquista de ranking com participantes suficientes	Aprovado
TU68	Unitário	Não conceder conquista fora da posição da meta	Aprovado
TU69	Unitário	Desbloquear conquista em cascata pelo bônus	Aprovado
TU70	Unitário	Calcular primeira data em dia diferente da semana	Aprovado
TU71	Unitário	Manter data no mesmo dia com horário futuro	Aprovado
TU72	Unitário	Adiar data quando o horário já passou	Aprovado
TU73	Unitário	Avançar para a próxima semana em dia anterior	Aprovado
TU74	Unitário	Somar sete dias para a próxima ocorrência	Aprovado
TU75	Unitário	Retornar nulo sem ocorrências cadastradas	Aprovado
TU76	Unitário	Ignorar ocorrências inativas e validadas	Aprovado
TU77	Unitário	Retornar a ocorrência aberta mais antiga	Aprovado

9.2 Casos de Teste de Integração

Tabela 3 — Casos de teste de integração (TI01–TI28)

ID	Tipo	Objetivo	Resultado
TI01	Integração	Autenticar usuário com credenciais corretas	Aprovado
TI02	Integração	Rejeitar login com senha incorreta	Aprovado
TI03	Integração	Bloquear login de conta não confirmada	Aprovado
TI04	Integração	Rejeitar login com e-mail inexistente	Aprovado
TI05	Integração	Confirmar conta com token válido	Aprovado

TI06	Integração	Rejeitar confirmação com token inválido	Aprovado
TI07	Integração	Redefinir senha com token válido	Aprovado
TI08	Integração	Rejeitar redefinição com token inválido	Aprovado
TI09	Integração	Bloquear aluno em rota exclusiva de professor	Aprovado
TI10	Integração	Redirecionar usuário anônimo para login	Aprovado
TI11	Integração	Registrar check-in do aluno com sucesso	Aprovado
TI12	Integração	Bloquear check-in realizado por professor	Aprovado
TI13	Integração	Impedir check-in duplicado do aluno	Aprovado
TI14	Integração	Cancelar check-in já realizado	Aprovado
TI15	Integração	Bloquear check-in em treino lotado	Aprovado
TI16	Integração	Bloquear check-in em treino inativo	Aprovado
TI17	Integração	Criar treino avulso com sucesso	Aprovado
TI18	Integração	Editar treino próprio com sucesso	Aprovado
TI19	Integração	Bloquear edição de treino de outro professor (achado corrigido)	Aprovado
TI20	Integração	Remover treino próprio com sucesso	Aprovado
TI21	Integração	Bloquear remoção de treino de outro professor	Aprovado
TI22	Integração	Validar presença do aluno na ocorrência	Aprovado
TI23	Integração	Validar ausência do aluno na ocorrência	Aprovado
TI24	Integração	Impedir validação antes do horário do treino	Aprovado
TI25	Integração	Impedir validação duplicada da mesma ocorrência	Aprovado
TI26	Integração	Validar ocorrência sem participantes inscritos	Aprovado
TI27	Integração	Gerar nova ocorrência ao validar treino fixo	Aprovado
TI28	Integração	Encerrar ocorrência de treino avulso após validação	Aprovado

10 DESTAQUES DE CÓDIGO

Esta seção reserva espaço para a apresentação do código-fonte de 5 casos de teste unitários e 5 casos de teste de integração considerados mais representativos da suíte, selecionados por evidenciarem regras de negócio combinadas, casos de borda relevantes ou os achados descritos na seção 7.2.

10.1 Testes Unitários Seleccionados

TU40 — test_validar_token_expirado_retorna_none

Objetivo: Rejeitar token de e-mail expirado

```
TU-40

def test_validar_token_expirado_retorna_none(app):
    token = gerar_token("aluno@escolar.ifrn.edu.br", salt="email-confirm")

    # expiration negativo garante que o token já esteja expirado
    # no instante da validação, sem precisar esperar tempo real
    # transcorrer (evita teste lento/instável baseado em sleep()).
    email = validar_token(token, salt="email-confirm", expiration=-1)

    assert email is None
```

TU65 — test_verifica_conquista_elite_esportiva_exige_dez_avaliacoes

Objetivo: Exigir dez avaliações para a conquista de tier mais alto

```
TU-65

def test_verifica_conquista_elite_esportiva_exige_dez_avaliacoes(app, aluno, treino):
    # regra especial: "Elite esportiva!" exige 10 avaliações, não 3
    conquista = criar_conquista(
        tipo="media", meta=4.0, bonus=5, nome="Elite esportiva!"
    )

    for i, nota in enumerate([5, 5, 5, 5, 5, 5, 5, 5, 5, 5]): # 9 avaliações
        ocorrencia = criar_ocorrencia(treino, dias=i + 1)
        criar_frequencia(aluno, treino, ocorrencia, status="presente", nota=nota)

    verificar_conquistas(aluno.usr_id)

    assert not _concedida(aluno, conquista)

    # a décima avaliação completa o mínimo exigido
    ocorrencia = criar_ocorrencia(treino, dias=10)
    criar_frequencia(aluno, treino, ocorrencia, status="presente", nota=5)

    verificar_conquistas(aluno.usr_id)

    assert _concedida(aluno, conquista)
```

TU69 — test_verifica_conquista_cascata_pontos_bonus_desbloqueia_outra

Objetivo: Desbloquear conquista em cascata pelo bônus de outra

```
TU-69

def test_verifica_conquista_cascata_pontos_bonus_desbloqueia_outra(app, aluno):
    """
    Ganhar pontos por uma conquista pode ser suficiente pra desbloquear
    outra na mesma chamada (a função se chama recursivamente quando
    houve_nova é True).
    """
    conquista_a = criar_conquista(
        tipo="pontos", meta=50, bonus=60, nome="A", tier_valor=1
    )
    conquista_b = criar_conquista(
        tipo="pontos", meta=100, bonus=10, nome="B", tier_valor=2
    )

    aluno.usr_pontos = 50 # atinge só a meta de A (50 + 60 = 110 ≥ meta de B)
    db.session.commit()

    verificar_conquistas(aluno.usr_id)

    assert _concedida(aluno, conquista_a)
    assert _concedida(aluno, conquista_b)
    assert aluno.usr_pontos == 120
```

TU72 — test_primeira_data_mesmo_dia_horario_ja_passou

Objetivo: Adiar o cálculo de data quando o horário já passou

```
TU-72

def test_primeira_data_mesmo_dia_horario_ja_passou(monkeypatch):
    # "Agora" é segunda-feira, às 20h
    agora = datetime(2026, 7, 20, 20, 0)
    _freeze(monkeypatch, agora)

    # Pede segunda-feira às 10h (já passou hoje) → deve ir pra próxima semana
    resultado = calcular_primeira_data(0, time(10, 0))

    assert resultado == datetime(2026, 7, 27, 10, 0)
```

TU33 — test_ja_fez_checkin_true_com_inscricao *Objetivo: Retornar verdadeiro para aluno já inscrito*

```
TU-33

def test_ja_fez_checkin_true_com_inscricao(app, aluno, treino, ocorrencia):
    with app.test_request_context():
        criar_frequencia(aluno, treino, ocorrencia)

        login_user(aluno)

        assert treino.ja_fez_checkin is True

        logout_user()
```

10.2 Testes de Integração Selecionados

TI03 — test_login_conta_nao_confirmada

Objetivo: Bloquear login de conta não confirmada

```
TI-03

def test_login_conta_nao_confirmada(client):
    criar_aluno(
        email="naoconfirmado@escolar.ifrn.edu.br",
        confirmed=False
    )

    response = client.post(
        "/auth/login",
        data={"email": "naoconfirmado@escolar.ifrn.edu.br", "senha": "12345678"},
        follow_redirects=True
    )

    assert response.status_code == 200
    assert "confirmar seu email".encode("utf-8") in response.data
```

TI09 — test_professor_required_bloqueia_aluno

Objetivo: Bloquear aluno em rota exclusiva de professor

```
TI-09

def test_professor_required_bloqueia_aluno(client, login_aluno):
    response = client.get("/treinos/criar")

    assert response.status_code == 403
```


TI19 — test_editar_treino_de_outro_professor_e_bloqueado

Objetivo: Confirmar a correção do achado de autorização na edição de treino

```
TI-19

def test_editar_treino_de_outro_professor_e_bloqueado(client, login_professor, modalidade):
    outro_professor = User(
        usr_nome="Outro Professor",
        usr_email="outroprof@ifrn.edu.br",
        usr_senha_hash=generate_password_hash("12345678"),
        usr_tipo="professor",
        usr_confirmed=True,
        usr_is_active=True,
        usr_primeiro_login=False,
        usr_pontos=0,
        usr_img="images/default_profile.png"
    )
    db.session.add(outro_professor)
    db.session.commit()

    treino_do_outro = criar_treino(outro_professor, modalidade, nome="Treino do Outro Professor")

    response = client.post(
        f"/treinos/editar/{treino_do_outro.trn_id}",
        data=_payload_avulso(modalidade, nome="Nome Alterado Por Outro Professor")
    )

    assert response.status_code == 404

    treino_intacto = Treino.query.get(treino_do_outro.trn_id)
    assert treino_intacto.trn_nome == "Treino do Outro Professor"
```

TI21 — test_remover_treino_de_outro_professor_e_bloqueado

Objetivo: Confirmar o bloqueio de remoção entre professores

```
TI-21

def test_remover_treino_de_outro_professor_e_bloqueado(client, login_professor, modalidade):
    outro_professor = User(
        usr_nome="Outro Professor",
        usr_email="outroprof2@ifrn.edu.br",
        usr_senha_hash=generate_password_hash("12345678"),
        usr_tipo="professor",
        usr_confirmed=True,
        usr_is_active=True,
        usr_primeiro_login=False,
        usr_pontos=0,
        usr_img="images/default_profile.png"
    )
    db.session.add(outro_professor)
    db.session.commit()

    treino_do_outro = criar_treino(outro_professor, modalidade, nome="Treino Protegido")

    response = client.post(f"/treinos/deletar/{treino_do_outro.trn_id}")

    assert response.status_code == 404

    treino_intacto = Treino.query.get(treino_do_outro.trn_id)
    assert treino_intacto.trn_ativo is True
```

TI27 — test_treino_fixo_cria_nova_ocorrencia

Objetivo: Gerar nova ocorrência ao validar treino fixo

```
● ● ● TI-27

def test_treino_fixo_cria_nova_ocorrencia(
    client,
    login_professor,
    treino,
    ocorrencia,
    frequencia
):
    treino.trn_fixo = True

    ocorrencia.tro_data = datetime.now() - timedelta(hours=1)

    db.session.commit()

    response = client.post(
        f"/treinos/detalhes/{ocorrencia.tro_id}/validar",
        data={
            f"status_{frequencia.frq_id}": "on"
        },
        follow_redirects=False
    )

    assert response.status_code == 302

    quantidade = TreinoOcorrencia.query.filter_by(
        tro_treino_id=treino.trn_id
    ).count()

    assert quantidade == 2

    nova_ocorrencia = (
        TreinoOcorrencia.query
        .filter_by(tro_treino_id=treino.trn_id)
        .order_by(TreinoOcorrencia.tro_data.desc())
        .first()
    )

    assert nova_ocorrencia.tro_id != ocorrencia.tro_id
    assert nova_ocorrencia.tro_data > ocorrencia.tro_data
    assert nova_ocorrencia.tro_ativo is True
```

11 CONCLUSÃO

A elaboração da suíte de testes automatizados e do teste de sistema para o IFSports permitiu não apenas verificar a corretude das regras de negócio já implementadas, como também identificar pontos de melhoria no próprio código-fonte, evidenciando o valor dos testes como ferramenta de qualidade que vai além da simples confirmação de comportamento esperado. A combinação de 77 testes unitários, 28 testes de integração e a validação manual gravada em vídeo cobriu, dentro do prazo disponível, os módulos de maior criticidade do sistema — autenticação, autorização, ciclo de vida de treinos e gamificação — sem a necessidade de esgotar a cobertura de módulos de menor risco. As lacunas de cobertura identificadas ficam registradas como direcionamento para a continuidade deste trabalho. Dessa forma, conclui-se que o conjunto de testes elaborado fornece evidências suficientes de que os principais requisitos funcionais do IFSports foram implementados corretamente e apresentam comportamento consistente frente aos cenários previstos.

12 REFERÊNCIAS

PYTEST. **Pytest documentation**. Disponível em: <https://docs.pytest.org>. Acesso em: 21 jul. 2026.

PALLETS PROJECTS. **Flask documentation**. Disponível em: <https://flask.palletsprojects.com>. Acesso em: 21 jul. 2026.

PALLETS PROJECTS. **Flask-SQLAlchemy documentation**. Disponível em: <https://flask-sqlalchemy.palletsprojects.com>. Acesso em: 21 jul. 2026.

SINGUREANU, Constantin. **O que são testes de sistema? Um mergulho profundo nas abordagens, tipos, ferramentas, dicas e truques, e muito mais!** Disponível em: <https://www.zaptest.com/pt-br/o-que-sao-testes-de-sistema-um-mergulho-profundo-nas-abordagens-tipos-ferramentas-dicas-e-truques-e-muito-mais>. Acesso em: 21 jul. 2026.

VIEIRA, Higor. **Teste unitário: entenda a importância desse tipo de teste para o desenvolvimento de software**. Disponível em: <https://www.objective.com.br/insights/teste-unitario/>. Acesso em: 21 jul. 2026.

VIEIRA, Higor. **Testes de integração: conheça os tipos e as vantagens em automatizar.**
Disponível em: <https://www.objective.com.br/insights/teste-de-integracao/>. Acesso em: 21 jul.
2026b.